formalmind

WETH9

Formal Verification Audit

Formal Mind

Table of Contents

1. Summary . . . **2**

2. Functions . . . **3**

2.1. `deposit` . . . 3

2.2. `receive / fallback` . . . 3

2.3. `withdraw` . . . 4

2.4. `totalSupply` . . . 5

2.5. `approve` . . . 5

2.6. `transfer` . . . 5

2.7. `transferFrom` . . . 6

3. Statements . . . **8**

3.1. S-01: Solvency: the sum of all balances never exceeds the ETH the contract holds, after any call. . . . 8

3.2. S-02: totalSupply() can be larger than the sum of balances: ETH forced in (selfdestruct, coinbase) is counted but owned by no one. . . . 9

3.3. S-03: transfer and transferFrom conserve the sum of balances, including a transfer to yourself. . . . 11

3.4. S-04: No one can lower another account's balance except through an allowance that account gave them, and an allowance of uint(-1) is never spent down. . . . 12

3.5. S-05: Anyone holding a balance can always withdraw all of it (no call by another account can stop them), unless their own address rejects ETH sent with 2300 gas. . . . 13

3.6. S-06: The sum of `WETH9.balanceOf` over all accounts equals `totalSupply()`, i.e. `address(this).balance`, apart from ETH forced in (selfdestruct or coinbase): without forced ETH it is exact equality, and with it the sum is lower by exactly the forced amount. . . . 15

3.7. S-08: `WETH9.transfer` and `WETH9.transferFrom` change `balanceOf` only at `src` and `dst`: every other account's `balanceOf` is unchanged (and `allowance` changes only at `[src][msg.sender]` in `transferFrom`). . . . 17

4. The bytecode . . . **19**

5. Scope and trusted base . . . **20**

Contract: `src/WETH9.sol`, canonical Wrapped Ether (Solidity `>=0.4.22 <0.6`)
Repository: formalmind/weth9 at `fa35972`
Method: Formal Mind's agent swarm proves each function and each statement in Lean 4 against a model formal-ast generates from solc's AST; every theorem is checked by Lean and approved by a reviewer agent. Two statements are also carried to the compiled bytecode through EquiVM's refinement proof.
Auditor: Formal Mind

1. Summary

- **7 of 7 functions proved**: when each reverts, what it writes, what it returns.
- **7 of 7 statements proved** about the whole contract, among them solvency.
- **Solvency and transfer conservation hold of the bytecode**, through EquiVM (one stated assumption for `withdraw`).
- **No findings.**

Check it yourself:

```
cd formal && lake build        # the source-level proofs
cd bytecode && lake build      # the bytecode proofs (EquiVM, pinned)
```

2. Functions

2.1. deposit

```
function deposit() public payable {
    balanceOf[msg.sender] += msg.value;
    emit Deposit(msg.sender, msg.value);
}
```

Proved in formal/Proofs/Swarm/WETH9Deposit.lean:

```
theorem deposit_spec {σ : Type} (ext : Ext σ) (env : Env) (w : σ) :
    ((WETH9.deposit ext).run env).run w =
      .ok ((), ext.set w { ext.get w with
        balanceOf := upd (ext.get w).balanceOf env.sender
          (addw 256 ((ext.get w).balanceOf env.sender) env.value) }) := …
```

```
theorem deposit_frame {σ : Type} (ext : Ext σ) (env : Env) (w w' : σ)
    (hgs : ∀ w s, ext.get (ext.set w s) = s)
    (h : ((WETH9.deposit ext).run env).run w = .ok ((), w')) :
    (ext.get w').balanceOf env.sender = (addw 256 ((ext.get w).balanceOf env.sender) env.value) ∧
    (∀ a, a ≠ env.sender → (ext.get w').balanceOf a = (ext.get w).balanceOf a) ∧
    (ext.get w').allowance = (ext.get w).allowance ∧
    (ext.get w').name = (ext.get w).name ∧
    (ext.get w').symbol = (ext.get w).symbol ∧
    (ext.get w').decimals = (ext.get w).decimals := …
```

2.2. receive / fallback

```
function() external payable {
    deposit();
}
```

Proved in formal/Proofs/Swarm/WETH9Fallback.lean:

```
theorem fallback_spec {σ : Type} (ext : Ext σ) (env : Env) (w : σ) :
    ((WETH9.«» ext).run env).run w =
      .ok ((), ext.set w { ext.get w with
        balanceOf := upd (ext.get w).balanceOf env.sender
```

```
          (addw 256 ((ext.get w).balanceOf env.sender) env.value) }) := …
```

```
theorem fallback_frame {σ : Type} (ext : Ext σ) (env : Env) (w w' : σ)
    (hgs : ∀ w s, ext.get (ext.set w s) = s)
    (h : ((WETH9.«» ext).run env).run w = .ok ((), w')) :
    (ext.get w').balanceOf env.sender = (addw 256 ((ext.get w).balanceOf env.sender) env.value) ∧
    (∀ a, a ≠ env.sender → (ext.get w').balanceOf a = (ext.get w).balanceOf a) ∧
    (ext.get w').allowance = (ext.get w).allowance ∧
    (ext.get w').name = (ext.get w).name ∧
    (ext.get w').symbol = (ext.get w).symbol ∧
    (ext.get w').decimals = (ext.get w).decimals := …
```

2.3. withdraw

```
function withdraw(uint wad) public {
    require(balanceOf[msg.sender] >= wad);
    balanceOf[msg.sender] -= wad;
    msg.sender.transfer(wad);
    emit Withdrawal(msg.sender, wad);
}
```

Proved in formal/Proofs/Swarm/WETH9Withdraw.lean:

```
theorem withdraw_reverts {σ : Type} (ext : Ext σ) (wad : Nat) (env : Env) (w : σ)
    (h : (ext.get w).balanceOf env.sender < wad) :
    ((WETH9.withdraw ext wad).run env).run w = .error ⟨"require"⟩ := …
```

```
theorem withdraw_spec {σ : Type} (ext : Ext σ) (wad : Nat) (env : Env) (w : σ)
    (h : wad ≤ (ext.get w).balanceOf env.sender) :
    ((WETH9.withdraw ext wad).run env).run w =
      (match ((ext.lowCall env.sender wad []).run env).run
          (ext.set w { ext.get w with
            balanceOf := upd (ext.get w).balanceOf env.sender
              (subw 256 ((ext.get w).balanceOf env.sender) wad) }) with
        | .error e => .error e
        | .ok (r, w2) => if r.1 then .ok ((), w2) else .error ⟨"transfer"⟩) := …
```

```
theorem withdraw_sub_exact (bal wad : Nat) (h : wad ≤ bal) (hb : bal < 2 ^ 256) :
    subw 256 bal wad = bal - wad := …
```

2.4. totalSupply

```
function totalSupply() public view returns (uint) {
    return address(this).balance;
}
```

Proved in formal/Proofs/Swarm/WETH9TotalSupply.lean:

```
theorem WETH9.totalSupply_spec {σ : Type} (ext : Ext σ) (env : Env) (s : σ) :
    ((WETH9.totalSupply ext).run env).run s = .ok (env.balance, s) := rfl
```

2.5. approve

```
function approve(address guy, uint wad) public returns (bool) {
    allowance[msg.sender][guy] = wad;
    emit Approval(msg.sender, guy, wad);
    return true;
}
```

Proved in formal/Proofs/Swarm/WETH9Approve.lean:

```
theorem approve_spec {σ : Type} (ext : Ext σ) (guy wad : Nat) (env : Env) (w : σ) :
    ((WETH9.approve ext guy wad).run env).run w =
      .ok (true, ext.set w { ext.get w with
        allowance := upd (ext.get w).allowance env.sender
          (upd ((ext.get w).allowance env.sender) guy wad) }) := …
```

```
theorem approve_allowance {σ : Type} (ext : Ext σ) (guy wad : Nat) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s) (o sp : Nat) :
    ∃ w' : σ, ((WETH9.approve ext guy wad).run env).run w = .ok (true, w') ∧
      (ext.get w').allowance o sp =
        (if o = env.sender ∧ sp = guy then wad else (ext.get w).allowance o sp) ∧
      (ext.get w').balanceOf = (ext.get w).balanceOf := …
```

2.6. transfer

```
function transfer(address dst, uint wad) public returns (bool) {
    return transferFrom(msg.sender, dst, wad);
}
```

Proved in formal/Proofs/Swarm/WETH9Transfer.lean:

```
theorem transfer_reverts {σ : Type} (ext : WETH9.Ext σ) (dst wad : Nat) (e : Env) (w : σ)
    (h : (ext.get w).balanceOf e.sender < wad) :
    ((WETH9.transfer ext dst wad).run e).run w = .error ⟨"require"⟩ := …
```

```
theorem transfer_succeeds {σ : Type} (ext : WETH9.Ext σ) (dst wad : Nat) (e : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (h : wad ≤ (ext.get w).balanceOf e.sender) :
    ((WETH9.transfer ext dst wad).run e).run w =
      .ok (true, ext.set w
        (let s := ext.get w
         let b1 := upd s.balanceOf e.sender (subw 256 (s.balanceOf e.sender) wad)
         { s with balanceOf := upd b1 dst (addw 256 (b1 dst) wad) })) := …
```

2.7. transferFrom

```
function transferFrom(address src, address dst, uint wad)
    public
    returns (bool)
{
    require(balanceOf[src] >= wad);

    if (src != msg.sender && allowance[src][msg.sender] != uint(-1)) {
        require(allowance[src][msg.sender] >= wad);
        allowance[src][msg.sender] -= wad;
    }

    balanceOf[src] -= wad;
    balanceOf[dst] += wad;

    emit Transfer(src, dst, wad);

    return true;
}
```

Proved in formal/Proofs/Swarm/WETH9TransferFrom.lean:

```
theorem transferFrom_reverts_balance {σ : Type} (ext : WETH9.Ext σ) (src dst wad : Nat) (e : Env) (w :
σ)
    (h : (ext.get w).balanceOf src < wad) :
    ((WETH9.transferFrom ext src dst wad).run e).run w = .error ⟨"require"⟩ := …
```

```
theorem transferFrom_reverts_allowance {σ : Type} (ext : WETH9.Ext σ) (src dst wad : Nat) (e : Env) (w :
σ)
    (hb : wad ≤ (ext.get w).balanceOf src)
    (hs : src ≠ e.sender)
    (hm : (ext.get w).allowance src e.sender ≠ 2 ^ 256 - 1)
    (ha : (ext.get w).allowance src e.sender < wad) :
    ((WETH9.transferFrom ext src dst wad).run e).run w = .error ⟨"require"⟩ := …
```

```
theorem transferFrom_succeeds_spend {σ : Type} (ext : WETH9.Ext σ) (src dst wad : Nat) (e : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (hb : wad ≤ (ext.get w).balanceOf src)
    (hs : src ≠ e.sender)
    (hm : (ext.get w).allowance src e.sender ≠ 2 ^ 256 - 1)
    (ha : wad ≤ (ext.get w).allowance src e.sender) :
    ((WETH9.transferFrom ext src dst wad).run e).run w =
      .ok (true, ext.set w
        (let s := ext.get w
         let al := upd s.allowance src
           (upd (s.allowance src) e.sender (subw 256 (s.allowance src e.sender) wad))
         let b1 := upd s.balanceOf src (subw 256 (s.balanceOf src) wad)
         { s with allowance := al, balanceOf := upd b1 dst (addw 256 (b1 dst) wad) })) := …
```

```
theorem transferFrom_succeeds_nospend {σ : Type} (ext : WETH9.Ext σ) (src dst wad : Nat) (e : Env) (w :
σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (hb : wad ≤ (ext.get w).balanceOf src)
    (hn : src = e.sender ∨ (ext.get w).allowance src e.sender = 2 ^ 256 - 1) :
    ((WETH9.transferFrom ext src dst wad).run e).run w =
      .ok (true, ext.set w
        (let s := ext.get w
         let b1 := upd s.balanceOf src (subw 256 (s.balanceOf src) wad)
         { s with balanceOf := upd b1 dst (addw 256 (b1 dst) wad) })) := …
```

3. Statements

3.1. S-01: Solvency: the sum of all balances never exceeds the ETH the contract holds, after any call.

Status: proved (invariant).

Precisely: In every reachable state of WETH9, the sum of `balanceOf[a]` over all addresses a is at most `address(this).balance`. It holds at deployment (both zero). It keeps holding after every successful call to `deposit`, the receive/fallback, `withdraw`, `transfer`, `transferFrom` and `approve`, including when `withdraw's ETH send re-enters the contract.

Assumes:

- ETH forced in without a call (selfdestruct, coinbase rewards) only raises the contract's balance, so it can only widen the gap. It is allowed, and the claim is ≤ rather than =.
- The sum of balances is taken over the naturals (unbounded), not in uint256 with wrap-around.
- Calls that revert leave the state unchanged, so they can't break the invariant.

The theorems (`formal/Proofs/Statements/S01`), one per function:

```
theorem S01.WETH9_approve {σ : Type} (ext : WETH9.Ext σ) (guy wad : Nat) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hinv : ∀ S : List Nat, S.Nodup → (S.map (ext.get w).balanceOf).sum ≤ env.balance) :
    ∃ w' : σ, ((WETH9.approve ext guy wad).run env).run w = .ok (true, w') ∧
      ∀ S : List Nat, S.Nodup → (S.map (ext.get w').balanceOf).sum ≤ env.balance := …
```

```
theorem S01.WETH9_deposit {σ : Type} (ext : WETH9.Ext σ) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hval : env.value ≤ env.balance)
    (hinv : ∀ S : List Nat, S.Nodup →
      (S.map (ext.get w).balanceOf).sum ≤ env.balance - env.value) :
    ∃ w' : σ, ((WETH9.deposit ext).run env).run w = .ok ((), w') ∧
      ∀ S : List Nat, S.Nodup → (S.map (ext.get w').balanceOf).sum ≤ env.balance := …
```

```
theorem S01.WETH9_fallback {σ : Type} (ext : WETH9.Ext σ) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hval : env.value ≤ env.balance)
    (hinv : ∀ S : List Nat, S.Nodup →
      (S.map (ext.get w).balanceOf).sum ≤ env.balance - env.value) :
    ∃ w' : σ, ((WETH9.«» ext).run env).run w = .ok ((), w') ∧
```

```
      ∀ S : List Nat, S.Nodup → (S.map (ext.get w').balanceOf).sum ≤ env.balance := …
```

```
theorem S01.WETH9_transfer {σ : Type} (ext : WETH9.Ext σ) (dst wad : Nat) (e : Env)
    (w w' : σ) (r : Bool)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (hsa : e.sender < 2 ^ 160) (hda : dst < 2 ^ 160)
    (hbal : ∀ a, (ext.get w).balanceOf a < 2 ^ 256)
    (hnoov : dst ≠ e.sender → (ext.get w).balanceOf dst + wad < 2 ^ 256)
    (hinv : S03T.total (ext.get w).balanceOf (2 ^ 160) ≤ e.balance)
    (hok : ((WETH9.transfer ext dst wad).run e).run w = .ok (r, w')) :
    S03T.total (ext.get w').balanceOf (2 ^ 160) ≤ e.balance := …
```

```
theorem S01.WETH9_transferFrom {σ : Type} (ext : WETH9.Ext σ) (src dst wad : Nat) (e : Env)
    (w w' : σ) (r : Bool)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (hsA : src < 2 ^ 160) (hdA : dst < 2 ^ 160)
    (hs : (ext.get w).balanceOf src < 2 ^ 256)
    (hd : src ≠ dst → (ext.get w).balanceOf dst + wad < 2 ^ 256)
    (hinv : WETH9.Swarm.Cons.lsum (ext.get w).balanceOf (List.range (2 ^ 160)) ≤ e.balance)
    (hok : ((WETH9.transferFrom ext src dst wad).run e).run w = .ok (r, w')) :
    WETH9.Swarm.Cons.lsum (ext.get w').balanceOf (List.range (2 ^ 160)) ≤ e.balance := …
```

```
theorem S01.WETH9_withdraw {σ : Type} (ext : WETH9.Ext σ) (wad : Nat) (env : Env) (w w' : σ)
    (ethBal : σ → Nat)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hbal : ∀ w s, ethBal (ext.set w s) = ethBal w)
    (hrange : (ext.get w).balanceOf env.sender < 2 ^ 256)
    (hcall : ∀ w1 r w2, ((ext.lowCall env.sender wad []).run env).run w1 = .ok (r, w2) →
      (∀ S : List Nat, S.Nodup → (S.map (ext.get w1).balanceOf).sum + wad ≤ ethBal w1) →
      ∀ S : List Nat, S.Nodup → (S.map (ext.get w2).balanceOf).sum ≤ ethBal w2)
    (hinv : ∀ S : List Nat, S.Nodup → (S.map (ext.get w).balanceOf).sum ≤ ethBal w)
    (hok : ((WETH9.withdraw ext wad).run env).run w = .ok ((), w')) :
    ∀ S : List Nat, S.Nodup → (S.map (ext.get w').balanceOf).sum ≤ ethBal w' := …
```

3.2. S-02: totalSupply() can be larger than the sum of balances: ETH forced in (selfdestruct, coinbase) is counted but owned by no one.

Status: proved (invariant).

Precisely: In every reachable state of WETH9, the sum of `balanceOf` over all accounts is at most `totalSupply()` (the contract's ETH balance, `address(this).balance`). Every entry point

(receive/fallback, deposit, withdraw, transfer, transferFrom, approve) keeps this true. Equality is not guaranteed: ETH that reaches the contract without calling it (selfdestruct to it, coinbase rewards, ETH sent before deployment) raises `totalSupply()` but credits no `balanceOf`. So the gap `totalSupply() - sum(balanceOf)` is never negative and can be positive.

Assumes:

- totalSupply() returns address(this).balance
- Forced ETH is modelled as an environment step that raises the contract's balance and changes no storage
- The ETH that withdraw sends out leaves the contract's balance exactly by wad, and a reentrant call during that send also keeps the invariant
- No uint256 overflow of balanceOf entries (sum of balances is bounded by total ETH)

The theorems (`formal/Proofs/Statements/S02`), one per function:

```
theorem S02.WETH9_approve {σ : Type} (ext : WETH9.Ext σ) (guy wad : Nat) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hinv : ∀ S : List Nat, S.Nodup → (S.map (ext.get w).balanceOf).sum ≤ env.balance) :
    ∃ w' : σ, ((WETH9.approve ext guy wad).run env).run w = .ok (true, w') ∧
      ∀ S : List Nat, S.Nodup → (S.map (ext.get w').balanceOf).sum ≤ env.balance := …
```

```
theorem S02.WETH9_deposit {σ : Type} (ext : WETH9.Ext σ) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hval : env.value ≤ env.balance)
    (hinv : ∀ S : List Nat, S.Nodup →
      (S.map (ext.get w).balanceOf).sum ≤ env.balance - env.value) :
    ∃ w' : σ, ((WETH9.deposit ext).run env).run w = .ok ((), w') ∧
      ∀ S : List Nat, S.Nodup → (S.map (ext.get w').balanceOf).sum ≤ env.balance := …
```

```
theorem S02.WETH9_fallback {σ : Type} (ext : WETH9.Ext σ) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hval : env.value ≤ env.balance)
    (hinv : ∀ S : List Nat, S.Nodup →
      (S.map (ext.get w).balanceOf).sum ≤ env.balance - env.value) :
    ∃ w' : σ, ((WETH9.«» ext).run env).run w = .ok ((), w') ∧
      ∀ S : List Nat, S.Nodup → (S.map (ext.get w').balanceOf).sum ≤ env.balance := …
```

```
theorem S02.WETH9_transfer {σ : Type} (ext : WETH9.Ext σ) (dst wad : Nat) (e : Env)
    (w w' : σ) (r : Bool)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
```

```
    (hsa : e.sender < 2 ^ 160) (hda : dst < 2 ^ 160)
    (hbal : ∀ a, (ext.get w).balanceOf a < 2 ^ 256)
    (hnoov : dst ≠ e.sender → (ext.get w).balanceOf dst + wad < 2 ^ 256)
    (hinv : S03T.total (ext.get w).balanceOf (2 ^ 160) ≤ e.balance)
    (hok : ((WETH9.transfer ext dst wad).run e).run w = .ok (r, w')) :
    S03T.total (ext.get w').balanceOf (2 ^ 160) ≤ e.balance := …
```

```
theorem S02.WETH9_transferFrom {σ : Type} (ext : WETH9.Ext σ) (src dst wad : Nat) (e : Env)
    (w w' : σ) (r : Bool)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (hsA : src < 2 ^ 160) (hdA : dst < 2 ^ 160)
    (hs : (ext.get w).balanceOf src < 2 ^ 256)
    (hd : src ≠ dst → (ext.get w).balanceOf dst + wad < 2 ^ 256)
    (hinv : WETH9.Swarm.Cons.lsum (ext.get w).balanceOf (List.range (2 ^ 160)) ≤ e.balance)
    (hok : ((WETH9.transferFrom ext src dst wad).run e).run w = .ok (r, w')) :
    WETH9.Swarm.Cons.lsum (ext.get w').balanceOf (List.range (2 ^ 160)) ≤ e.balance := …
```

```
theorem S02.WETH9_withdraw {σ : Type} (ext : WETH9.Ext σ) (wad : Nat) (env : Env) (w w' : σ)
    (ethBal : σ → Nat)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hbal : ∀ w s, ethBal (ext.set w s) = ethBal w)
    (hrange : (ext.get w).balanceOf env.sender < 2 ^ 256)
    (hcall : ∀ w1 r w2, ((ext.lowCall env.sender wad []).run env).run w1 = .ok (r, w2) →
      (∀ S : List Nat, S.Nodup → (S.map (ext.get w1).balanceOf).sum + wad ≤ ethBal w1) →
      ∀ S : List Nat, S.Nodup → (S.map (ext.get w2).balanceOf).sum ≤ ethBal w2)
    (hinv : ∀ S : List Nat, S.Nodup → (S.map (ext.get w).balanceOf).sum ≤ ethBal w)
    (hok : ((WETH9.withdraw ext wad).run env).run w = .ok ((), w')) :
    ∀ S : List Nat, S.Nodup → (S.map (ext.get w').balanceOf).sum ≤ ethBal w' := …
```

3.3. S-03: transfer and transferFrom conserve the sum of balances, including a transfer to yourself.

Status: proved (conservation).

Precisely: For WETH9.transfer and WETH9.transferFrom only: after any call that doesn't revert, the total of balanceOf over all addresses is the same as before. This also holds when the sender and the recipient are the same address. Reverted calls change nothing. deposit and withdraw are not covered.

Assumes:

- The sum is taken over every address (all 2^160 of them), as a mathematical total of balanceOf
- Only successful calls are considered; a reverting call leaves state unchanged

- transfer is treated as transferFrom(msg.sender, dst, wad), as WETH9 implements it

The theorems (`formal/Proofs/Statements/S03`), one per function:

```
theorem S03.WETH9_transfer {σ : Type} (ext : WETH9.Ext σ) (dst wad : Nat) (e : Env)
    (w w' : σ) (r : Bool)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (hsa : e.sender < 2 ^ 160) (hda : dst < 2 ^ 160)
    (hbal : ∀ a, (ext.get w).balanceOf a < 2 ^ 256)
    (hnoov : dst ≠ e.sender → (ext.get w).balanceOf dst + wad < 2 ^ 256)
    (hok : ((WETH9.transfer ext dst wad).run e).run w = .ok (r, w')) :
    S03T.total (ext.get w').balanceOf (2 ^ 160) = S03T.total (ext.get w).balanceOf (2 ^ 160) := …
```

```
theorem S03.WETH9_transferFrom {σ : Type} (ext : WETH9.Ext σ) (src dst wad : Nat) (e : Env)
    (w w' : σ) (r : Bool)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (hsA : src < 2 ^ 160) (hdA : dst < 2 ^ 160)
    (hs : (ext.get w).balanceOf src < 2 ^ 256)
    (hd : src ≠ dst → (ext.get w).balanceOf dst + wad < 2 ^ 256)
    (hok : ((WETH9.transferFrom ext src dst wad).run e).run w = .ok (r, w')) :
    WETH9.Swarm.Cons.lsum (ext.get w').balanceOf (List.range (2 ^ 160))
      = WETH9.Swarm.Cons.lsum (ext.get w).balanceOf (List.range (2 ^ 160)) := …
```

3.4. S-04: No one can lower another account's balance except through an allowance that account gave them, and an allowance of uint(-1) is never spent down.

Status: proved (function).

Precisely: Whenever a caller successfully moves a nonzero amount out of someone else's balance with transferFrom, src must have approved that caller for at least that amount beforehand. The allowance then goes down by exactly the amount moved, except an allowance of type(uint256).max, which stays the same. The other functions that write balanceOf (deposit, withdraw, transfer) are taken to change only the caller's own balance, or to raise someone else's. That part is an assumption here and is not proved.

Assumes:

- deposit, withdraw and transfer never lower the balance of an account other than msg.sender (transfer delegates to transferFrom with src = msg.sender)

- Transfers to oneself (src == dst) and zero-amount transfers don't count as lowering a balance
- uint(-1) means type(uint256).max

The theorems (`formal/Proofs/Statements/S04`), one per function:

```
theorem S04.WETH9_transferFrom {σ : Type} (ext : WETH9.Ext σ) (src dst wad : Nat) (e : Env)
    (w w' : σ) (r : Bool)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (hs : src ≠ e.sender) (_hd : src ≠ dst) (_hpos : 0 < wad)
    (hwad : wad < 2 ^ 256)
    (hrange : (ext.get w).allowance src e.sender < 2 ^ 256)
    (hok : ((WETH9.transferFrom ext src dst wad).run e).run w = .ok (r, w')) :
    wad ≤ (ext.get w).allowance src e.sender ∧
    (ext.get w').allowance src e.sender =
      (if (ext.get w).allowance src e.sender = 2 ^ 256 - 1
       then (ext.get w).allowance src e.sender
       else (ext.get w).allowance src e.sender - wad) := …
```

3.5. S-05: Anyone holding a balance can always withdraw all of it (no call by another account can stop them), unless their own address rejects ETH sent with 2300 gas.

Status: proved (sequence).

Precisely: Take any reachable state of WETH9 and any address A with balanceOf[A] = b. After any sequence of calls by other accounts (deposit, withdraw, transfer, transferFrom, approve), A can still call withdraw(balanceOf[A]) and it succeeds: A's balance drops to 0 and A receives that much ETH. Other accounts can't lower balanceOf[A] without A's approval, and they can't drain the contract's ETH below the total of all balances. The only allowed failure is A's own address reverting or running out of the 2300 gas when `transfer` sends it ETH. This rests on the fact that WETH9's ETH balance is always at least the sum of balanceOf (solvency).

Assumes:

- A has given no allowance, or 'another account' excludes spenders A has approved: an approved spender can move A's balance with transferFrom
- The call has enough gas, and the call depth limit is not reached
- Solvency: address(WETH9).balance >= sum of balanceOf in every reachable state. ETH forced in through selfdestruct only makes this easier to hold
- A's balance can grow through deposits or incoming transfers, and 'all of it' means A's balance at the

moment A withdraws

The theorems (`formal/Proofs/Statements/S05`), one per function:

```
theorem S05.WETH9_approve {σ : Type} (ext : WETH9.Ext σ) (guy wad : Nat) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s) (A : Nat) (hA : env.sender ≠ A) :
    ∃ w' : σ, ((WETH9.approve ext guy wad).run env).run w = .ok (true, w') ∧
      (ext.get w').balanceOf = (ext.get w).balanceOf ∧
      (ext.get w').allowance A = (ext.get w).allowance A := …
```

```
theorem S05.WETH9_deposit {σ : Type} (ext : WETH9.Ext σ) (env : Env) (w : σ) (A : Nat)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hA : env.sender ≠ A)
    (hval : env.value ≤ env.balance)
    (hinv : ∀ S : List Nat, S.Nodup →
      (S.map (ext.get w).balanceOf).sum ≤ env.balance - env.value) :
    ∃ w' : σ, ((WETH9.deposit ext).run env).run w = .ok ((), w') ∧
      (ext.get w').balanceOf A = (ext.get w).balanceOf A ∧
      (ext.get w').allowance = (ext.get w).allowance ∧
      ∀ S : List Nat, S.Nodup → (S.map (ext.get w').balanceOf).sum ≤ env.balance := …
```

```
theorem S05.WETH9_fallback {σ : Type} (ext : WETH9.Ext σ) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s) (A : Nat) (hA : env.sender ≠ A)
    (hinv : ∀ S : List Nat, S.Nodup →
      (S.map (ext.get w).balanceOf).sum ≤ env.balance - env.value)
    (hval : env.value ≤ env.balance) :
    ∃ w' : σ, ((WETH9.«» ext).run env).run w = .ok ((), w') ∧
      (ext.get w').balanceOf A = (ext.get w).balanceOf A ∧
      (ext.get w').allowance = (ext.get w).allowance ∧
      ∀ S : List Nat, S.Nodup → (S.map (ext.get w').balanceOf).sum ≤ env.balance := …
```

```
theorem S05.WETH9_transfer {σ : Type} (ext : WETH9.Ext σ) (dst wad : Nat) (e : Env)
    (w w' : σ) (r : Bool)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (A : Nat) (hA : e.sender ≠ A)
    (hnoov : (ext.get w).balanceOf A + wad < 2 ^ 256)
    (hok : ((WETH9.transfer ext dst wad).run e).run w = .ok (r, w')) :
    (ext.get w).balanceOf A ≤ (ext.get w').balanceOf A ∧
      (ext.get w').allowance A = (ext.get w).allowance A := …
```

```
theorem S05.WETH9_transferFrom {σ : Type} (ext : WETH9.Ext σ) (src dst wad : Nat) (e : Env)
    (w w' : σ) (r : Bool) (A : Nat)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
```

```
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (hA : A ≠ e.sender)
    (hnoappr : (ext.get w).allowance A e.sender = 0)
    (hrangeA : (ext.get w).balanceOf A < 2 ^ 256)
    (hnoover : (ext.get w).balanceOf A + wad < 2 ^ 256)
    (hok : ((WETH9.transferFrom ext src dst wad).run e).run w = .ok (r, w')) :
    (ext.get w).balanceOf A ≤ (ext.get w').balanceOf A := …
```

```
theorem S05.WETH9_withdraw {σ : Type} (ext : WETH9.Ext σ) (wad : Nat) (env : Env) (w w' : σ)
    (A : Nat) (ethBal : σ → Nat)
    (hA : env.sender ≠ A)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hbal : ∀ w s, ethBal (ext.set w s) = ethBal w)
    (hrange : (ext.get w).balanceOf env.sender < 2 ^ 256)
    (hcallA : ∀ w1 r w2, ((ext.lowCall env.sender wad []).run env).run w1 = .ok (r, w2) →
      (ext.get w1).balanceOf A ≤ (ext.get w2).balanceOf A)
    (hcall : ∀ w1 r w2, ((ext.lowCall env.sender wad []).run env).run w1 = .ok (r, w2) →
      (∀ S : List Nat, S.Nodup → (S.map (ext.get w1).balanceOf).sum + wad ≤ ethBal w1) →
      ∀ S : List Nat, S.Nodup → (S.map (ext.get w2).balanceOf).sum ≤ ethBal w2)
    (hinv : ∀ S : List Nat, S.Nodup → (S.map (ext.get w).balanceOf).sum ≤ ethBal w)
    (hok : ((WETH9.withdraw ext wad).run env).run w = .ok ((), w')) :
    (ext.get w).balanceOf A ≤ (ext.get w').balanceOf A ∧
    ∀ S : List Nat, S.Nodup → (S.map (ext.get w').balanceOf).sum ≤ ethBal w' := …
```

3.6. S-06: The sum of `WETH9.balanceOf` over all accounts equals `totalSupply()`, i.e. `address(this).balance`, apart from ETH forced in (selfdestruct or coinbase): without forced ETH it is exact equality, and with it the sum is lower by exactly the forced amount.

Status: proved (invariant).

Precisely: In every reachable state of WETH9, the sum of balanceOf[a] over all addresses a plus the ETH forced into the contract (via selfdestruct or as a coinbase/block reward, which runs no code) equals address(this).balance, which is what totalSupply() returns. This must hold after every state-changing entry point (deposit and the receive/fallback path, withdraw, transfer, transferFrom, approve). If no ETH was ever forced in, the sum of balances is exactly totalSupply(). The forced amount is the total ETH received without going through deposit or the fallback.

Assumes:

- Forced ETH is modelled as ghost state: the running total of ETH that arrived without calling deposit or the fallback
- withdraw's ETH transfer to msg.sender either succeeds in full or reverts the whole call (re-entry

during the send leaves the relation intact)

- The contract starts with zero balance and zero balanceOf everywhere, apart from ETH already forced in, which counts toward the forced amount

The theorems (`formal/Proofs/Statements/S06`), one per function:

```
theorem S06.WETH9_approve {σ : Type} (ext : WETH9.Ext σ) (guy wad : Nat) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s) (forced : Nat) (S : List Nat)
    (hS : ∀ a, a ∉ S → (ext.get w).balanceOf a = 0)
    (hinv : (S.map (ext.get w).balanceOf).sum + forced = env.balance) :
    ∀ (r : Bool) (w' : σ), ((WETH9.approve ext guy wad).run env).run w = .ok (r, w') →
      (∀ a, a ∉ S → (ext.get w').balanceOf a = 0) ∧
      (S.map (ext.get w').balanceOf).sum + forced = env.balance := …
```

```
theorem S06.WETH9_deposit {σ : Type} (ext : WETH9.Ext σ) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s) (forced : Nat) (S : List Nat)
    (hnd : S.Nodup) (hsender : env.sender ∈ S)
    (hval : env.value ≤ env.balance) (hbal : env.balance < 2 ^ 256)
    (hS : ∀ a, a ∉ S → (ext.get w).balanceOf a = 0)
    (hinv : (S.map (ext.get w).balanceOf).sum + forced = env.balance - env.value) :
    ∀ (r : Unit) (w' : σ), ((WETH9.deposit ext).run env).run w = .ok (r, w') →
      (∀ a, a ∉ S → (ext.get w').balanceOf a = 0) ∧
      (S.map (ext.get w').balanceOf).sum + forced = env.balance := …
```

```
theorem S06.WETH9_fallback {σ : Type} (ext : WETH9.Ext σ) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s) (forced : Nat) (S : List Nat)
    (hnd : S.Nodup) (hsender : env.sender ∈ S)
    (hval : env.value ≤ env.balance) (hbal : env.balance < 2 ^ 256)
    (hS : ∀ a, a ∉ S → (ext.get w).balanceOf a = 0)
    (hinv : (S.map (ext.get w).balanceOf).sum + forced = env.balance - env.value) :
    ∀ (r : Unit) (w' : σ), ((WETH9.«» ext).run env).run w = .ok (r, w') →
      (∀ a, a ∉ S → (ext.get w').balanceOf a = 0) ∧
      (S.map (ext.get w').balanceOf).sum + forced = env.balance := …
```

```
theorem S06.WETH9_transfer {σ : Type} (ext : WETH9.Ext σ) (dst wad : Nat) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (hbal : env.balance < 2 ^ 256)
    (forced : Nat) (S : List Nat) (hnd : S.Nodup)
    (hS : ∀ a, a ∉ S → (ext.get w).balanceOf a = 0)
    (hinv : (S.map (ext.get w).balanceOf).sum + forced = env.balance) :
    ∀ (r : Bool) (w' : σ), ((WETH9.transfer ext dst wad).run env).run w = .ok (r, w') →
      ∃ S' : List Nat, S'.Nodup ∧
        (∀ a, a ∉ S' → (ext.get w').balanceOf a = 0) ∧
```

```
        (S'.map (ext.get w').balanceOf).sum + forced = env.balance := …
```

```
theorem S06.WETH9_transferFrom {σ : Type} (ext : WETH9.Ext σ) (src dst wad : Nat) (env : Env)
    (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t)
    (hbal : env.balance < 2 ^ 256)
    (forced : Nat) (S : List Nat) (hnd : S.Nodup)
    (hS : ∀ a, a ∉ S → (ext.get w).balanceOf a = 0)
    (hinv : (S.map (ext.get w).balanceOf).sum + forced = env.balance) :
    ∀ (r : Bool) (w' : σ), ((WETH9.transferFrom ext src dst wad).run env).run w = .ok (r, w') →
      ∃ S' : List Nat, S'.Nodup ∧
        (∀ a, a ∉ S' → (ext.get w').balanceOf a = 0) ∧
        (S'.map (ext.get w').balanceOf).sum + forced = env.balance := …
```

```
theorem S06.WETH9_withdraw {σ : Type} (ext : WETH9.Ext σ) (wad : Nat) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s) (forced : Nat) (S : List Nat)
    (hnd : S.Nodup) (hsender : env.sender ∈ S)
    (hrange : (ext.get w).balanceOf env.sender < 2 ^ 256)
    (hcall : ∀ (w1 w2 : σ) r, ((ext.lowCall env.sender wad []).run env).run w1 = .ok (r, w2) →
      (ext.get w2).balanceOf = (ext.get w1).balanceOf)
    (hS : ∀ a, a ∉ S → (ext.get w).balanceOf a = 0)
    (hinv : (S.map (ext.get w).balanceOf).sum + forced = env.balance) :
    ∀ (r : Unit) (w' : σ), ((WETH9.withdraw ext wad).run env).run w = .ok (r, w') →
      (∀ a, a ∉ S → (ext.get w').balanceOf a = 0) ∧
      (S.map (ext.get w').balanceOf).sum + forced = env.balance - wad ∧ wad ≤ env.balance := …
```

3.7. S-08: `WETH9.transfer` and `WETH9.transferFrom` change `balanceOf` only at `src` and `dst`: every other account's `balanceOf` is unchanged (and `allowance` changes only at `[src][msg.sender]` in `transferFrom`).

Status: proved (conservation).

Precisely: For any call to `WETH9.transfer(dst, wad)` or `WETH9.transferFrom(src, dst, wad)` that does not revert, every address other than `src` and `dst` has the same `balanceOf` after the call as before. For `transfer`, `src` is `msg.sender`. Nothing is claimed about `allowance`, about the new balances of `src` and `dst`, or about calls that revert.

Assumes:

- For transfer, src is msg.sender (transfer delegates to transferFrom(msg.sender, dst, wad)).

- Only successful (non-reverting) calls are covered; a revert leaves all state unchanged anyway.
- src == dst is allowed; then only that one account may change (it actually nets to zero).

The theorems (`formal/Proofs/Statements/S08`), one per function:

```
theorem S08.WETH9_transfer {σ : Type} (ext : WETH9.Ext σ) (dst wad : Nat) (env : Env) (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t) :
    ∀ (r : Bool) (w' : σ), ((WETH9.transfer ext dst wad).run env).run w = .ok (r, w') →
      ∀ a, a ≠ env.sender → a ≠ dst → (ext.get w').balanceOf a = (ext.get w).balanceOf a := …
```

```
theorem S08.WETH9_transferFrom {σ : Type} (ext : WETH9.Ext σ) (src dst wad : Nat) (env : Env)
    (w : σ)
    (hget : ∀ w s, ext.get (ext.set w s) = s)
    (hset : ∀ w s t, ext.set (ext.set w s) t = ext.set w t) :
    ∀ (r : Bool) (w' : σ), ((WETH9.transferFrom ext src dst wad).run env).run w = .ok (r, w') →
      ∀ a, a ≠ src → a ≠ dst → (ext.get w').balanceOf a = (ext.get w).balanceOf a := …
```

4. The bytecode

The theorems above are about the model formal-ast builds from the source, so they trust the compiler. These are about the compiled bytecode itself (`bytecode/`, on EquiVM pinned at `fc74a64`): `Ξ` is the EVM running WETH9's runtime bytecode, from any accounts, gas, caller, value and calldata.

```
theorem weth9_bytecode_conserves (hinj : SlotsInjective)
    {σ σ₀ σ' : AccountMap} {g g' : UInt256} {A A' : Substate} {I : ExecutionEnv} {o : ByteArray}
    (hcode : I.code = weth9Bytecode) (hsize : I.calldata.size < UInt256.size)
    (hsel : selIs I (weth9SelBytes 8) ∨ selIs I (weth9SelBytes 3))
    (hlt : totalBalAt σ I.codeOwner < UInt256.size)
    (hΞ : Ξ σ σ₀ g A I = .ok (.success (σ', g', A') o)) :
    totalBalAt σ' I.codeOwner = totalBalAt σ I.codeOwner := …
```

```
theorem weth9_bytecode_solvent_of_not_withdraw (hinj : SlotsInjective)
    {σ σ₀ σ' : AccountMap} {g g' : UInt256} {A A' : Substate} {I : ExecutionEnv} {o : ByteArray}
    (hcode : I.code = weth9Bytecode) (hsize : I.calldata.size < UInt256.size)
    (hwf : weth9StorageWF σ I) (hnw : ¬ selIs I (weth9SelBytes 4))
    (hpre : totalBalAt σ I.codeOwner + I.weiValue.toNat ≤ ethBalAt σ I.codeOwner)
    (hΞ : Ξ σ σ₀ g A I = .ok (.success (σ', g', A') o)) :
    totalBalAt σ' I.codeOwner ≤ ethBalAt σ' I.codeOwner := …
```

```
theorem weth9_bytecode_solvent_all (hinj : SlotsInjective)
    {σ σ₀ σ' : AccountMap} {g g' : UInt256} {A A' : Substate} {I : ExecutionEnv} {o : ByteArray}
    (hcode : I.code = weth9Bytecode) (hsize : I.calldata.size < UInt256.size)
    (hwf : weth9StorageWF σ I)
    (hcalls : CallsKeepSolvent I.codeOwner) (hself : HasCode σ I.codeOwner)
    (hpre : totalBalAt σ I.codeOwner + I.weiValue.toNat ≤ ethBalAt σ I.codeOwner)
    (hΞ : Ξ σ σ₀ g A I = .ok (.success (σ', g', A') o)) :
    totalBalAt σ' I.codeOwner ≤ ethBalAt σ' I.codeOwner := …
```

Assumptions: `SlotsInjective` (Keccak-256 collision resistance for the storage slots); for S-03, the balances sum below 2^256 before the call (solc 0.5 wraps `balanceOf[dst] + wad`); for S-01, solvency before the call and EquiVM's `weth9StorageWF` (`name` and `symbol` shorter than 2^64 bytes); for S-01 on `withdraw` only, `CallsKeepSolvent`: code that receives `withdraw`'s ETH, and calls back into WETH9, leaves it solvent. Details: bytecode/README.md.

5. Scope and trusted base

- Source-level proofs: Lean's kernel and its three standard axioms; formal-ast's translation of solc's AST into Lean.
- Bytecode proofs: also EquiVM's EVM semantics (passes the Ethereum conformance tests) and Lean's compiled evaluator for about 2,400 concrete facts (`native_decide`).
- Gas isn't modelled; `withdraw's `transfer` forwards 2300 gas, so a recipient that needs more can't withdraw (S-05 states this).
- ETH arriving without a call (selfdestruct, block rewards) is modelled as an increase in the contract's balance (S-02, S-06).
- `name`, `symbol` and `decimals` are constants, outside the proofs.

Exported from the proof ledger, `.formal/statements.md` and the Lean sources at `fa35972`.